Scalability: From 1 to N Projects I

The Standalone Project Paradigm enables horizontal scaling: adding a new project requires creating a directory with `manuscript/config.yaml` and nothing else. No infrastructure code changes, no `pyproject.toml` modifications, no CI configuration updates. The `run.sh` orchestrator automatically discovers new projects and presents them in its interactive menu.

We have validated scaling with 16 canonical exemplars under `projects/templates/`—always present for onboarding and tooling—and with this manuscript from `projects/templates/template_template` (130 tests) as a git-tracked public exemplar in the same automated discovery menus.

Canonical trio:

Scalability: From 1 to N Projects II

- **`template_code_project`**: Numerical optimization example with gradient-descent narration, 236 tests, 90%+ coverage. Minimal footprint: compact `src/`, scripted analysis, short manuscript sections.
- **`template_prose_project`**: Prose-heavy manuscript emphasizing narrative structure and bibliography discipline, 120 tests, 90%+ coverage—tests exercise rendering and Markdown integrity without heavyweight numerics.
- **`template_autoresearch_project`**: AutoResearch readiness workflow invoking `projects/templates/template_search_project/scripts/` to run corpus builders, scripted figures (`output/figures/`), and manifold-variable injection (the literature-search exemplar). Typical Stage 02 workloads include bibliography fusion, corpus JSON assembly, deep-search aggregates, and report composition.

Scalability: From 1 to N Projects III

Meta manuscript (**`projects/templates/template_template`**) analyzes the repository via `src/template_template/` introspection metrics; it now lives alongside the other public exemplars under `projects/templates/`.

These workspaces share no project-level code—only Layer 1 (23 infrastructure subdirectories, ~616 Python files)—validating insulation between domain repos and reusable services.

Scalability: From 1 to N Projects IV

Multi-Project Orchestration

When the `--all-projects` flag is passed to `run.sh`, the pipeline executes each discovered project sequentially, running infrastructure tests once at the start and skipping them for individual projects to avoid redundant validation. After all projects complete, a cross-project executive report aggregates metrics (test counts, coverage percentages, page counts, rendering durations) into a unified dashboard with both JSON and Markdown output formats. This executive reporting stage provides repository-level visibility without requiring any project-specific reporting code.

Scalability: From 1 to N Projects V

Scaling Metrics

Metric	`template_code_project`	`template_prose_project`	`template_autoresearch_project`
Source modules	26	6	60
Test files	12	8	17
Test count	236	120	296
Manuscript chapters	9	8	6
Analysis scripts	7	4	4
Figures (auto-generated)	9	5	27

The infrastructure overhead per project is constant regardless of project size: the same 23 modules, the same 13 pipeline stages, the same rendering and validation logic. This O(1) infrastructure cost is the architectural payoff of the Two-Layer separation.